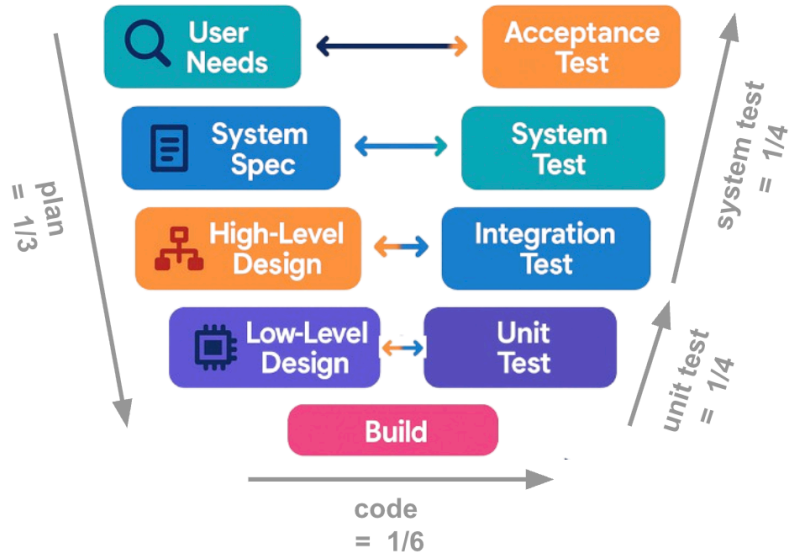
Home Syllabus Teams1 Teams2 One Two Chat Vids © timm 2025

CSC510: Software Engineering
NC State, Fall '25

Q1: does vibe code == pain maintain?

Q2: How much could/should AI change this "standard model"?



This subject: work in groups; write **quality** code; maintain alien code; use SE + AI; establish on-line profile. (For more on above pic, see Fig3 of Long et.al, TSE'23.)

CSC510: Requirements

- When
- Famous Requirements Failures
  - Ariane 5 Rocket Explosion 1996
  - London Ambulance Service Failure 1992
  - Heathrow Terminal 5 Baggage System 2008
  - Denver Airport Baggage System 1995
  - Mars Climate Orbiter 1999
  - Venus Rocket Explosion 1962
  - Genesis Spacecraft 2004
  - Stardust Spacecraft 2006
- Stakeholders in Requirements Engineering
  - Who Matters
  - Power Interest Grid
  - Elicitation and Management
- Requirement Interactions
  - Functional Requirements Interactions
    - Positive Interactions
    - Negative Interactions

    - Non Functional Requirements Interactions
        - Positive Interactions
        - Negative Interactions
- Requirements Process Models
    - Plan-Driven Models
    - Agile and Scrum
- Elicitation Techniques
    - Interviews and Observation
    - CRC Cards
    - Repertory Grids
- Requirements Traceability
- Requirements in Contracts and Procurement
- Requirements Quality Metrics

When

Recall the V-diagram:

- As we approach coding, we plan expectations that guide testing.
- Leaving coding, testing feeds back into refining requirements.

Requirements are a pre-code activity defining *what needs to be done*.

- Requirements engineering works when we can generate tests.
- Testing works when it can update requirements.

Famous Requirements Failures

Ariane 5 Rocket Explosion 1996

The Ariane 5 rocket exploded 37 seconds after launch due to a software error in the inertial reference system. It reused code from Ariane 4 without accounting for differences in flight dynamics, violating requirements for adaptability and robustness.[1]

London Ambulance Service Failure 1992

A flawed computer-aided dispatch system, caused by poorly gathered requirements, couldn't handle real-world load and delayed ambulances—contributing to loss of lives.[2]

Heathrow Terminal 5 Baggage System 2008

Poorly defined requirements led to delays and baggage operation chaos, as stakeholder needs were insufficiently incorporated.[3]

Denver Airport Baggage System 1995

Incomplete requirements gathering and unrealistic assumptions led to massive delays, cost overruns, and system failure.[4]

Mars Climate Orbiter 1999

A mismatch in metric and imperial units, missing from explicit requirements, caused the spacecraft to disintegrate on entry.[5]

Venus Rocket Explosion 1962

A missing hyphen in guidance code (a tiny typo) caused misdirected flight and probe destruction—highlighting the need for precise requirements and testing.[6]

Genesis Spacecraft 2004

An inverted accelerometer went unnoticed due to missing integration testing requirements, leading to mission failure upon re-entry.[7]

Stardust Spacecraft 2006

Lessons learned from Genesis led to well-specified testing and requirements validation—resulting in a successful comet sample return.[8]

Stakeholders in Requirements Engineering

Who Matters

Without stakeholders, requirements are just ideas. Key groups include:

- **Users** (e.g., travelers, students)
- **Clients/Sponsors** (e.g., executives, deans)

- **Domain Experts** (e.g., pilots, academic advisors)
- **Developers/Engineers**
- **Regulators/Legal** (e.g., FAA, accreditation bodies)
- **Indirect/Negative** (e.g., baggage handlers, alumni)

Power Interest Grid

A 2×2 matrix guides engagement strategy:

Power \ Interest	High Interest	Low Interest
High Power	Manage closely	Keep satisfied
Low Power	Keep informed	Monitor

Elicitation & Management

- **Identification**: Brainstorm stakeholder register.
- **Engagement**: Tailor communication by grid quadrant.
- **Traceability**: Link every requirement to a stakeholder.
- **Conflict Resolution**: Use mapping to reconcile competing needs.

Requirement Interactions

Functional Requirements Interactions

Req ID	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
R1=Auth		–								
R2=Search	+				+					
R3=Booking				–		+				
R4=Passenger Mgmt			–							
R5=Payment		+							–	
R6=Cancellation			+							
R7=Status Updates								+		

Req ID	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
R8=Seat Selection							+			–
R9=Loyalty					–					
R10=Multilang Support								–		

Positive Interactions

- R1&R2: Auth enables personalization in search
- R2&R5: Smooth flow into payment
- R3&R6: Booking and cancellation cohesion
- R7&R8: Live updates help seat choice
- R9&R10: Multi-language enhances loyalty programs

Negative Interactions

- R1&R2: Auth may slow search
- R3&R4: Conflicts if not synced
- R5&R9: Loyalty adds complexity to payment
- R8&R10: Multi-language complicates UI
- R9&R5: Loyalty and payment overlap issues

Non Functional Requirements Interactions

	A	C	I	M	P	Po	R	S	Se	U
A=Availability					+		+	+		–
C=Compliance			+	–					+	
I=Interoperability		+			–	+				
M=Maintainability		–			+		+			+
P=Performance	+		–	+		+	–	+		
Po=Portability			+		+			+		–
R=Reliability	+			+	–					

	A	C	I	M	P	Po	R	S	Se	U
S=Scalability	+				+	+				–
Se=Security		+		–						
U=Usability	–			+		–				

Positive Interactions

- Availability ↔ Performance, Reliability, Scalability
- Compliance ↔ Interoperability, Security
- Interoperability ↔ Compliance, Portability
- Maintainability ↔ Performance, Reliability, Usability
- Performance ↔ Availability, Reliability, Scalability, Portability
- Portability ↔ Interoperability, Performance, Scalability
- Reliability ↔ Availability, Performance
- Scalability ↔ Availability, Performance, Portability
- Security ↔ Compliance
- Usability ↔ Maintainability

Negative Interactions

- Availability ↔ Usability
- Compliance ↔ Maintainability
- Interoperability ↔ Performance
- Maintainability ↔ Compliance, Security
- Performance ↔ Interoperability, Reliability
- Portability ↔ Usability
- Reliability ↔ Performance
- Scalability ↔ Usability
- Security ↔ Maintainability
- Usability ↔ Availability, Portability

Requirements Process Models

Plan-Driven Models

Traditional methods like Waterfall and V-Model define requirements up front.
Ideal for stable, safety-critical domains (e.g., NASA Shuttle software following V-Model discipline).

Agile and Scrum

Agile is adaptive and iterative:

- Requirements formatted as **user stories**, prioritized in a backlog.
- Scrum roles: *Product Owner* (manages backlog), *Scrum Master*, *Development Team*.
- Requirements emerge and refine through sprints.
- **Example**: Spotify's squad model uses backlog-driven adaptation tied to user feedback loops.

Elicitation Techniques

Interviews and Observation

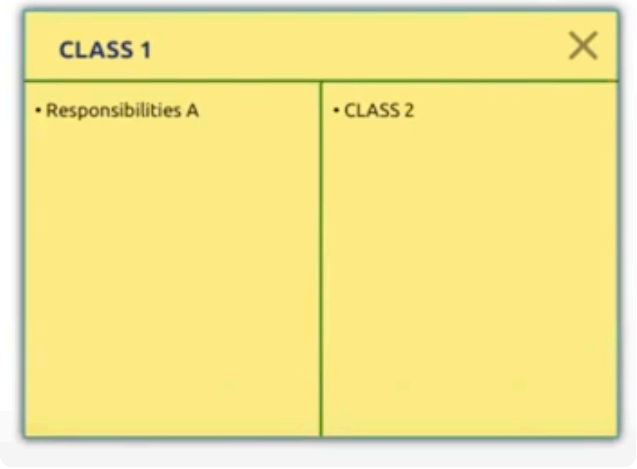
Classic techniques to gather requirements by surveying stakeholders and watching real workflows.

CRC Cards

Structured for object-oriented design:

- One index card per "class," divided into **Responsibilities** (left) and **Collaborators** (right).
- Supports brainstorming sessions and emergent architecture design.

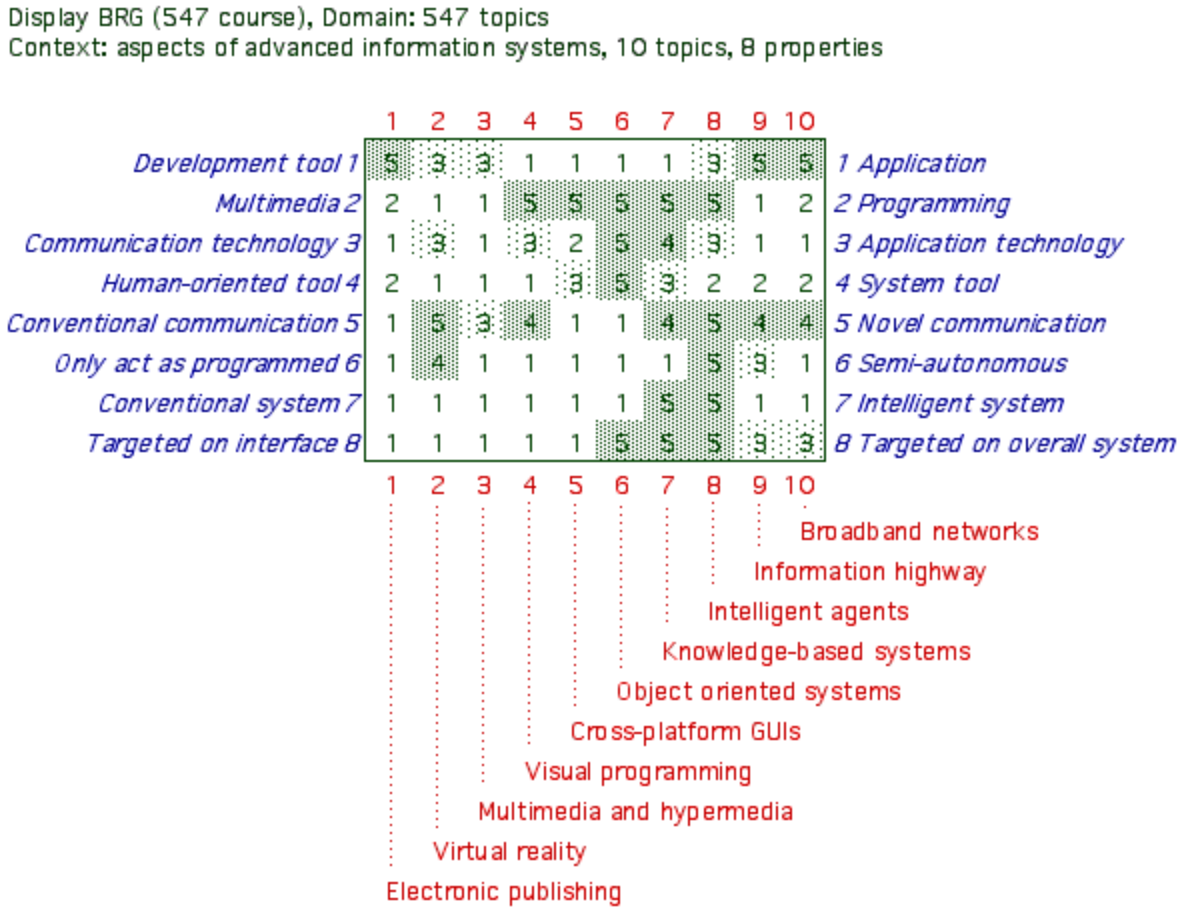
Example image of CRC cards:



Repertory Grids

A technique for eliciting personal constructs through comparisons:

Example image of a Repertory Grid:



Developed from psychology, this method reveals implicit requirements by asking stakeholders to compare sets and rate them on a bipolar scale. :contentReference[oaicite:3]{index=3}

1. A repertory grid is a **matrix** with elements (rows) and constructs (columns).
2. **Elements** = items being compared (e.g., tools, designs, products).
3. **Constructs** = bipolar distinctions (e.g., secure ↔ insecure).
4. Start by picking the **domain** and identifying relevant elements.
5. Then use the **triadic method**: find three things, then the dimension that separates two from the other.
6. This elicits new bipolar constructs to add as columns.
7. Stakeholders **rate each element** on each construct using a numeric scale.
8. The completed grid shows **patterns of similarity and difference**.
9. Analysis (clustering, factor analysis) reveals **groups of elements/constructs**.
10. Insights guide **requirements choices, prioritization, or design trade-offs**.

```
function cluster(elements):
    if size(elements) <= 1:
        return leaf(elements)
```

```
# Step 1: find most remote pair
(a, b) = argmax_distance(elements)

# Step 2: assign all elements to nearer of a or b
left, right = partition(elements, a, b)

# Step 3: recurse on each half
return node(
    cluster(left),
    cluster(right)
)
```

Requirements Traceability

Linking requirements through the lifecycle:

- **Forward**: requirement → design → implementation → test
- **Backward**: test or feature → original requirement
  Essential in regulated domains, e.g., DO-178C for avionics linking each test to a requirement.

Requirements in Contracts and Procurement

When outsourced, requirements become legally binding (via RFPs, proposals).
UK NHS IT failure (~£10bn loss) stemmed from ambiguous, loosely defined requirements in contracts.

Requirements Quality Metrics

Good requirements are **clear, correct, consistent, complete, feasible, testable, traceable** (IEEE 29148).

- **Poor**: "System must be fast."
- **Better**: "System shall process 95% of transactions in under 2 s under 10,000 concurrent users."
  In FDA-regulated medical systems, ambiguous requirements derail certification.